

SELLER AND INVENTORY MANAGEMENT

1.CREATE SELLER TABLE

Seller table stores the details of sellers, including Seller ID, Seller Name, Email, and GST Number.

Attribute	Data Type	Constraint
Seller_ID	NUMBER(5)	PRIMARY KEY
Seller_Name	VARCHAR2(50)	NOT NULL
Email	VARCHAR2(100)	UNIQUE
GST_Number	VARCHAR2(20)	UNIQUE

```
SQL> CREATE TABLE SELLER (  
2     SELLER_ID NUMBER(5) PRIMARY KEY,  
3     SELLER_NAME VARCHAR2(50) NOT NULL,  
4     EMAIL VARCHAR2(100) UNIQUE,  
5     GST_NUMBER VARCHAR2(20) UNIQUE  
6 );
```

Table created.

Insert Seller Table

```

SQL> INSERT INTO SELLER VALUES (101, 'ABC Electronics', 'abc@gmail.com', 'GST101');
1 row created.

SQL> INSERT INTO SELLER VALUES (102, 'Fashion World', 'fashion@gmail.com', 'GST102');
1 row created.

SQL> INSERT INTO SELLER VALUES (103, 'Tech Store', 'tech@gmail.com', 'GST103');
1 row created.

SQL> INSERT INTO SELLER VALUES (104, 'Smart Mart', 'smart@gmail.com', 'GST104');
1 row created.

SQL> INSERT INTO SELLER VALUES (105, 'Style Hub', 'style@gmail.com', 'GST105');
1 row created.

SQL> INSERT INTO SELLER VALUES (106, 'Digital World', 'digital@gmail.com', 'GST106');
1 row created.

SQL>
SQL> COMMIT;

Commit complete.

```

2.CREATE INVENTORY TABLE

Inventory table stores product stock details, including inventory ID, product ID, seller ID, stock quantity, and stock status.

Attribute	Data Type	Constraint
Inventory_ID	NUMBER(5)	PRIMARY KEY
Seller_ID	NUMBER(5)	FOREIGN KEY
Product_ID	NUMBER(5)	FOREIGN KEY
Stock_Quantity	NUMBER(10)	NOT NULL
Availability_Status	VARCHAR2(20)	NOT NULL

```
SQL> CREATE TABLE INVENTORY (  
2     INVENTORY_ID NUMBER(5) PRIMARY KEY,  
3     SELLER_ID NUMBER(5),  
4     PRODUCT_ID NUMBER(5),  
5     STOCK_QUANTITY NUMBER(5) NOT NULL,  
6     AVAILABILITY_STATUS VARCHAR2(20) NOT NULL,  
7     FOREIGN KEY (SELLER_ID) REFERENCES SELLER(SELLER_ID)  
8 );
```

Table created.

Insert Inventory Table

```
SQL> INSERT INTO INVENTORY VALUES (1, 101, 201, 25, 'Available');  
1 row created.  
  
SQL> INSERT INTO INVENTORY VALUES (2, 102, 202, 10, 'Available');  
1 row created.  
  
SQL> INSERT INTO INVENTORY VALUES (3, 103, 203, 0, 'Unavailable');  
1 row created.  
  
SQL> INSERT INTO INVENTORY VALUES (4, 104, 204, 30, 'Available');  
1 row created.  
  
SQL> INSERT INTO INVENTORY VALUES (5, 105, 205, 0, 'Unavailable');  
1 row created.  
  
SQL> INSERT INTO INVENTORY VALUES (6, 106, 206, 15, 'Available');  
1 row created.  
  
SQL>  
SQL> COMMIT;  
  
Commit complete.
```

3.Establish Relationships Between Sellers, Products, and Stock.

This query displays the details of sellers and their corresponding inventory information by joining the **SELLER** and **INVENTORY** tables using **SELLER_ID**. It shows the seller ID, seller name, inventory ID, product ID, stock quantity, and availability status.

```

SQL> SELECT
  2     S.SELLER_ID,
  3     S.SELLER_NAME,
  4     I.INVENTORY_ID,
  5     I.PRODUCT_ID,
  6     I.STOCK_QUANTITY,
  7     I.AVAILABILITY_STATUS
  8 FROM SELLER S
  9 JOIN INVENTORY I
 10 ON S.SELLER_ID = I.SELLER_ID;

```

SELLER_ID	SELLER_NAME	INVENTORY_ID
101	ABC Electronics	1
201	25 Available	
102	Fashion World	2
202	10 Available	
103	Tech Store	3
203	0 Unavailable	

PRODUCT_ID	STOCK_QUANTITY	AVAILABILITY_STATUS
104	Smart Mart	4
204	30 Available	
105	Style Hub	5
205	0 Unavailable	
106	Digital World	6
206	15 Available	

4.Maintain Seller Product Information:

JOIN operation was used to combine the Seller, Inventory, and Product tables. It retrieves the seller ID, seller name, product ID, product name, stock quantity, and availability status in a single result.

JOIN Query:

```

SELECT s.Seller_ID,
       s.Seller_Name,
       p.Product_ID,
       p.Product_Name,

```

```

        i.Stock_Quantity,
        i.Availability_Status
FROM Seller s
JOIN Inventory i
ON s.Seller_ID = i.Seller_ID
JOIN Product p
ON i.Product_ID = p.Product_ID;

```

```

SQL> SELECT s.Seller_ID,
2         s.Seller_Name,
3         p.Product_ID,
4         p.Product_Name,
5         i.Stock_Quantity,
6         i.Availability_Status
7 FROM Seller s
8 JOIN Inventory i
9 ON s.Seller_ID = i.Seller_ID
10 JOIN Product p
11 ON i.Product_ID = p.Product_ID;

```

SELLER_ID	SELLER_NAME	PRODUCT_ID	PRODUCT_NAME	STOCK_QUANTITY	AVAILABILITY_STATUS
101	New Electronics	201	Laptop	40	Available
102	Fashion World	202	Shirt	10	Available
103	Tech Store	203	Mobile	20	Available

5.Track Available and Unavailable Products

Input: Product_ID, Stock_Quantity

This operation is used to identify whether a product is **Available or Unavailable** based on its stock quantity. If the stock quantity is greater than 0, the product is considered **Available**; otherwise, it is considered **Unavailable**.

```
SQL> SELECT Product_ID,
2         Stock_Quantity,
3         'Unavailable' AS Availability_Status
4 FROM Inventory
5 WHERE Stock_Quantity = 0;
```

PRODUCT_ID	STOCK_QUANTITY	AVAILABILIT
205	0	Unavailable

```
SQL> SELECT Product_ID,
2         Stock_Quantity,
3         'Available' AS Availability_Status
4 FROM Inventory
5 WHERE Stock_Quantity > 0;
```

PRODUCT_ID	STOCK_QUANTITY	AVAILABIL
201	40	Available
202	10	Available
203	20	Available
204	30	Available
206	15	Available

6.Generate Inventory Status Reports

This operation is used to generate an inventory status report based on the current stock quantity. It identifies whether the product is **Available**, **Requires Reordering**, or **Unavailable**.

Input: Product_ID, Stock_Quantity, Availability_Status

```
SQL> SELECT Product_ID,
2         Stock_Quantity,
3         CASE
4             WHEN Stock_Quantity = 0 THEN 'Unavailable'
5             WHEN Stock_Quantity <= 10 THEN 'Requires Reordering'
6             ELSE 'Available'
7         END AS Inventory_Status
8 FROM Inventory;
```

PRODUCT_ID	STOCK_QUANTITY	INVENTORY_STATUS
201	40	Available
202	10	Requires Reordering
203	20	Available
204	30	Available
205	0	Unavailable
206	15	Available

6 rows selected.